

Hybrid Architecture Specification

Local First Data Analytics & LLM Routing for CDE Metrics

Project Overview

This document outlines the technical specification for a modern data estate designed to process California Department of Education (CDE) accountability logic. By combining a Local First Data Analytics pipeline with an LLM Routing Architecture, this system delivers auditable, complex metric reporting alongside conversational, natural language data exploration. It is scaled to manage analytics across a \$65 million educational asset portfolio effectively.

1. System Architecture & Stack

To ensure high performance, low cognitive load, and strict data governance, the system utilizes a keyboard-centric development workflow deploying to a local-first stack.

Layer	Technology / Strategy
Execution Engine	DuckDB & MotherDuck. Handles rapid, in-memory processing of complex multi-CTE queries.
Transformation	dbt (Data Build Tool). Materializes CDE business rules (attendance, EWS, suspensions) into standardized models.
Semantic Layer	One Big Table (OBT). Denormalized data marts exposed directly to the LLM to prevent SQL hallucination.
Visualization & UI	Rill / Custom Dashboards. Utilizing neuroaesthetic design principles to reduce cognitive load for site principals.
Language Model	LLM Router. Intercepts natural language queries and categorizes intent before execution.

2. Data Modeling Strategy (The OBT Approach)

Instead of requiring the NL2SQL model to dynamically generate complex joins across `stg_attendance`, `stg_discipline`, and `stg_grades`, the pipeline pre-calculates CDE metrics using dbt.

- **Staging Layer:** Raw, clean tables (e.g., `stg_enrollment`). Used for simple counting queries.

- **Marts Layer (OBTs):** Highly denormalized tables such as `mart_school_accountability` (containing chronic absenteeism and suspension rates) and `mart_student_composite_risk`.

Only these defined schemas are passed to the LLM context, effectively guiding the model to query the Semantic Layer rather than raw transactional tables.

3. LLM Router Implementation

The core innovation is the Router component, which intercepts user prompts from the chat interface and classifies them into one of three execution paths.

Path A: Simple Ad-Hoc (NL2SQL)

Trigger: Basic aggregation or filtering requests.

Example: *"How many active 8th graders are there?"*

Action: The LLM generates standard DuckDB SQL against the staging tables. Execution is passed directly to DuckDB.

Path B: Complex Metric Query (NL to Semantic Layer)

Trigger: Questions involving strictly defined CDE metrics or EWS flags.

Example: *"Which students have a composite risk score over 4?"*

Action: The router bypasses dynamic calculation. The LLM generates a simple `SELECT` statement against the pre-calculated `mart_student_composite_risk` OBT.

Path C: Dashboard Insight (Context-Aware RAG)

Trigger: Requests for summaries, "why" questions, or contextual analysis of the currently viewed dashboard.

Example: *"What does this spike in suspensions mean?"*

Action: SQL generation is bypassed entirely. The application passes the active JSON state of the dashboard to the LLM, which synthesizes a natural language response based on the visible data chunks.

4. Front-End Context Integration

To enable "Path C", the user interface must be context-aware. When a principal queries the chatbot, the front-end application silently appends a JSON payload containing the current application state. This reduces friction and allows the LLM to act as an on-screen analyst.

Context Payload Structure

```
{
  "user_query": "Why are 9th grade absences high?",
  "active_filters": {"school_id": "HS_01", "grade": "9"},
  "visible_data_summary": [...array of aggregated records currently rendered...]
}
```

5. Deployment Workflow

1. **Infrastructure Setup:** Configure DuckDB instances and initialize the dbt project using Zig-compiled extensions where necessary for performance.
2. **Pipeline Build:** Develop and test dbt models for CDE Chronic Absenteeism, Suspension rules, and EWS scoring.
3. **Router Logic:** Implement the Python/LangChain router to classify prompts and manage the schema context window.
4. **UI Development:** Build the frontend dashboard, ensuring the chat component captures and transmits the JSON state payload. Implement an interface that minimizes visual fatigue for educators.
5. **Stakeholder Review:** Present the unified data estate to IT operations and site leadership for operational feedback.